UNIVERSITY OF LAHORE - FACULTY OF INFORMATION TECHNOLOGY

Customer Churn Prediction Web Application Using Flask

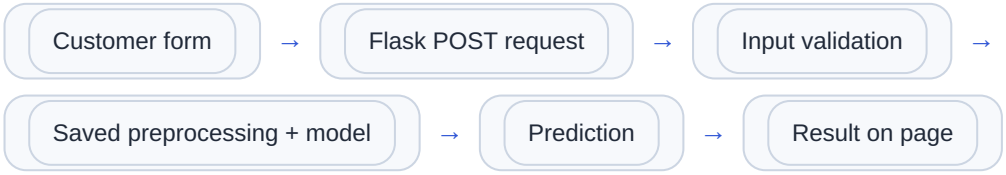
Artificial Intelligence - Assignment 2

Student Name	Abdullah Zahid
SAP ID	70172820
Section	5-G

Based on	Assignment 1 - Customer Churn Prediction Using Machine Learning
Final Model	Logistic Regression

1. Assignment 2 Overview

This assignment continues the customer churn work from Assignment 1. I used the same final Logistic Regression model and the same preprocessing pipeline, then connected it to a Flask web application. The page collects customer information and returns a clear **Churn** or **No Churn** result.



Assignment 1 model carried forward

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	80.55%	65.72%	55.88%	60.40%	84.19%

The full scikit-learn Pipeline is saved in one joblib file, so the numeric imputation, scaling, categorical imputation, one-hot encoding and model are kept together.

> **Why this matters:** The web application does not use hard-coded results. The customer values are passed to the same trained machine-learning pipeline used for Assignment 1.

2. Saving the Trained Model

I rebuilt the Assignment 1 Logistic Regression pipeline from the same churn dataset and saved it with **joblib**. The output below confirms the model metrics and the saved files.

Model Training and Saving - Executed Output

```
Assignment 1 final model: Logistic Regression
Dataset rows: 7043
Training rows: 5634 | Testing rows: 1409
Accuracy:  0.8055
Precision: 0.6572
Recall:    0.5588
F1-score:  0.6040
ROC-AUC:   0.8419
Saved model: model.joblib
Saved metadata: model_metadata.json
```

Screenshot 1 - Model saved successfully. The model file is `model.joblib`. A small metadata JSON file is also saved so the model metrics and feature order can be checked later.

3. Flask Project Structure

The files are separated into the backend, prediction logic, HTML template, CSS, trained model, testing script and deployment files. This keeps the project simple to understand and easy to upload to PythonAnywhere.

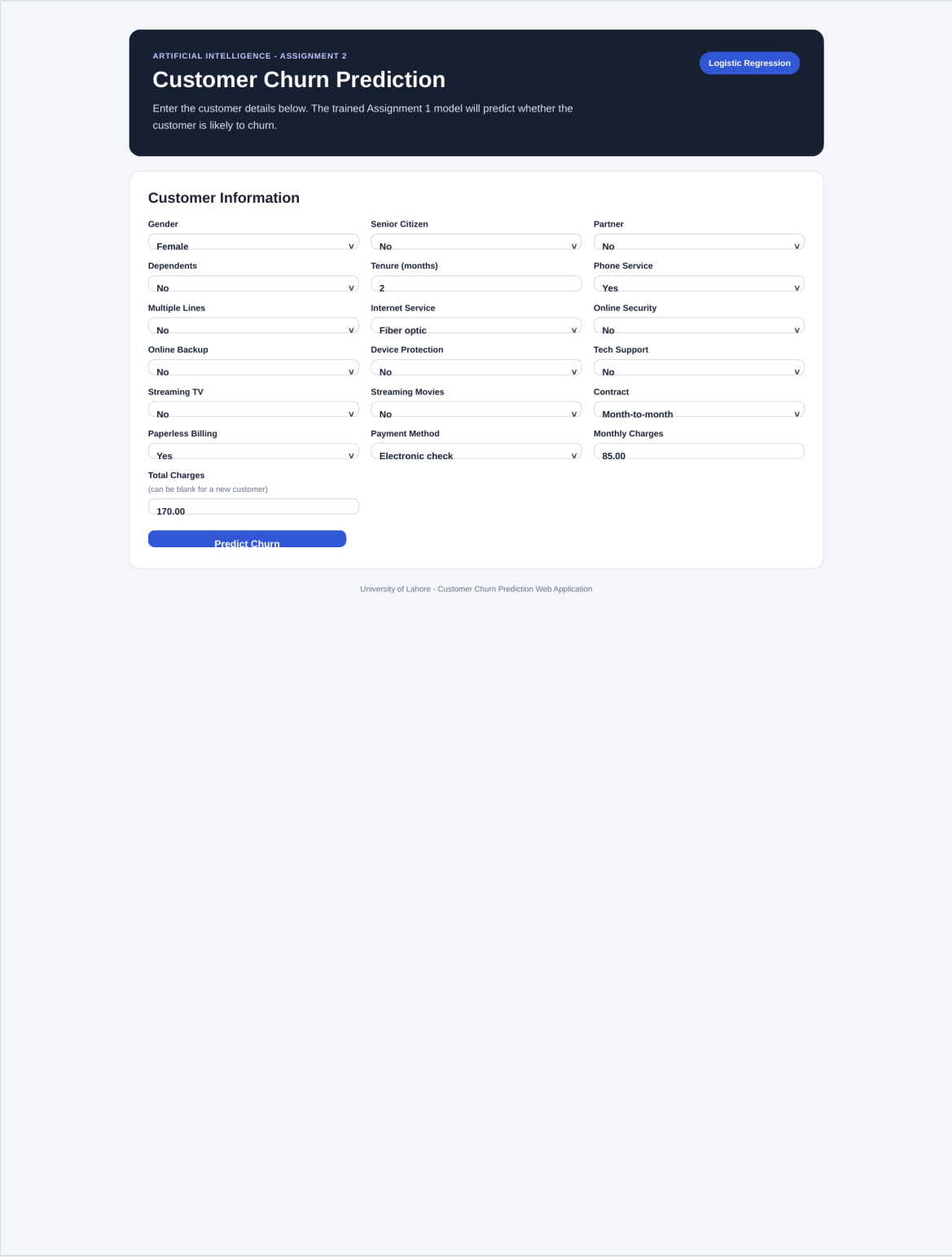
Flask Project Folder Structure

```
README.md
app.py
data/WA_Fn-UseC_-Telco-Customer-Churn.csv
deployment/pythonanywhere_wsgi.py.example
local_test.py
model.joblib
model_metadata.json
predictor.py
requirements.txt
static/style.css
templates/index.html
train_model.py
```

Screenshot 2 - Project folder structure. The required `app.py`, model file, `requirements.txt`, templates folder and static folder are present. Extra files were added for reproducibility and testing.

4. HTML and CSS Interface

The frontend contains all customer fields used by the trained model. Categorical values are entered through dropdowns and numeric values use number inputs. The layout is kept simple so the user can fill the form without seeing technical model details.



Screenshot 3 - Customer input form. The form includes contract, internet services, tenure, payment method, monthly charges and the other features expected by the saved pipeline.

5. Flask Backend and Prediction Logic

The main Flask route accepts both GET and POST requests. When the form is submitted, it calls the prediction function and returns the result to the same page. Errors are caught so incomplete or incorrect input does not crash the application.

Important Flask Backend Code - app.py

```
from flask import Flask, render_template, request
from predictor import make_prediction

app = Flask(__name__)


@app.route("/", methods=["GET", "POST"])
def index():
    result = None
    probability = None
    error = None

    if request.method == "POST":
        try:
            result, probability = make_prediction(request.form)
        except ValueError as exc:
            error = str(exc)
        except Exception:
            error = "Prediction could not be completed. Please check the entered values and try
again."

    return render_template(
        "index.html",
        result=result,
        probability=probability,
        error=error,

...
        form_data=request.form,
    )


if __name__ == "__main__":
    app.run(debug=True)

...
```

Screenshot 4 - Important code from app.py. The route receives the submitted form, calls `make_prediction()`, and sends the result or an error message back to the HTML template.

6. Input Conversion and Validation

The prediction helper keeps the feature order the same as Assignment 1. Numeric fields are converted to the correct type before the customer row is sent to the saved pipeline. It also checks required fields, tenure and charge values.

Prediction Logic and Input Validation - predictor.py

```
FEATURE_ORDER = [
    "gender", "SeniorCitizen", "Partner", "Dependents", "tenure",
    "PhoneService", "MultipleLines", "InternetService", "OnlineSecurity",
    "OnlineBackup", "DeviceProtection", "TechSupport", "StreamingTV",
    "StreamingMovies", "Contract", "PaperlessBilling", "PaymentMethod",
    "MonthlyCharges", "TotalCharges"
]


def customer_from_form(form):

    missing = [field for field in required_fields if not str(form.get(field, "")).strip()]
    if missing:
        raise ValueError("Please fill in all required fields.")

    try:
        senior_citizen = int(form["SeniorCitizen"])
        tenure = int(form["tenure"])
        monthly_charges = float(form["MonthlyCharges"])
        total_text = str(form.get("TotalCharges", "")).strip()
        total_charges = float(total_text) if total_text else np.nan
    except ValueError as exc:
        raise ValueError("Please enter valid numbers for tenure and charges.") from exc

    if senior_citizen not in (0, 1):
        raise ValueError("Senior Citizen must be Yes or No.")
    if tenure < 0 or tenure > 100:
        raise ValueError("Tenure must be between 0 and 100 months.")
    if monthly_charges < 0:
        raise ValueError("Monthly Charges cannot be negative.")
    if not np.isnan(total_charges) and total_charges < 0:
        raise ValueError("Total Charges cannot be negative.")

    values = {
        "gender": form["gender"],
        "SeniorCitizen": senior_citizen,
        "Partner": form["Partner"],
        "Dependents": form["Dependents"],
        "tenure": tenure,

...


def make_prediction(form):
    customer = customer_from_form(form)
    prediction = int(model.predict(customer)[0])
    probability = float(model.predict_proba(customer)[0, 1])
    return ("Churn" if prediction == 1 else "No Churn"), probability
```

Screenshot 5 - predictor.py. This code prepares the submitted customer data and calls the saved model. The prediction is converted to a readable Churn or No Churn label.

7. Local Prediction Testing

I tested two customers using the same prediction function used by the Flask application. One customer was designed as a high-risk case and the other as a stable long-term customer.

Local Model Test - Two Sample Customers

```
$ python local_test.py

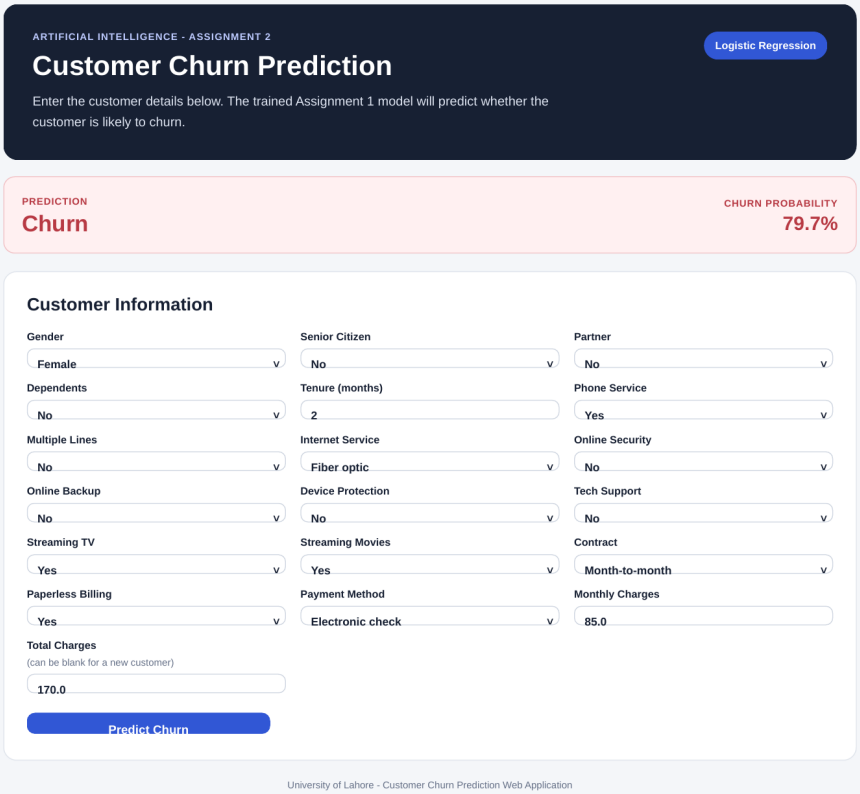
Customer 1: Churn | Churn probability: 0.7974
Customer 2: No Churn | Churn probability: 0.0068
```

Screenshot 6 - Local test results. Customer 1 is predicted as Churn with probability 0.7974. Customer 2 is predicted as No Churn with probability 0.0068.

> These results match the sample predictions from Assignment 1, which confirms that the saved model is being used consistently.

8. Churn Prediction Result

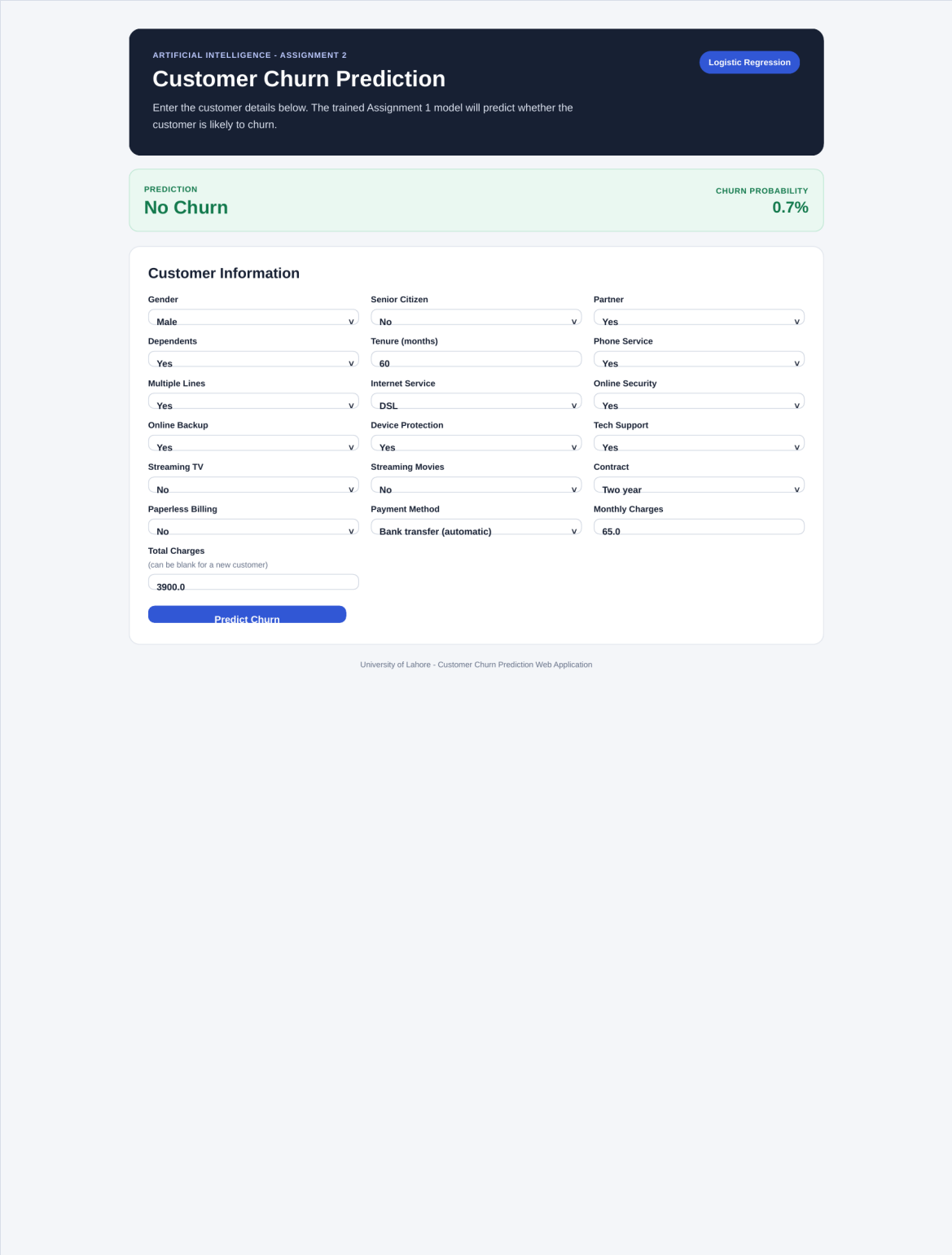
The first sample is a short-tenure, month-to-month Fiber Optic customer with higher monthly charges. The application shows a clear Churn result instead of a raw model array.



Screenshot 7 - Churn case. The result is **Churn** with a churn probability of about **79.7%**.

9. No Churn Prediction Result

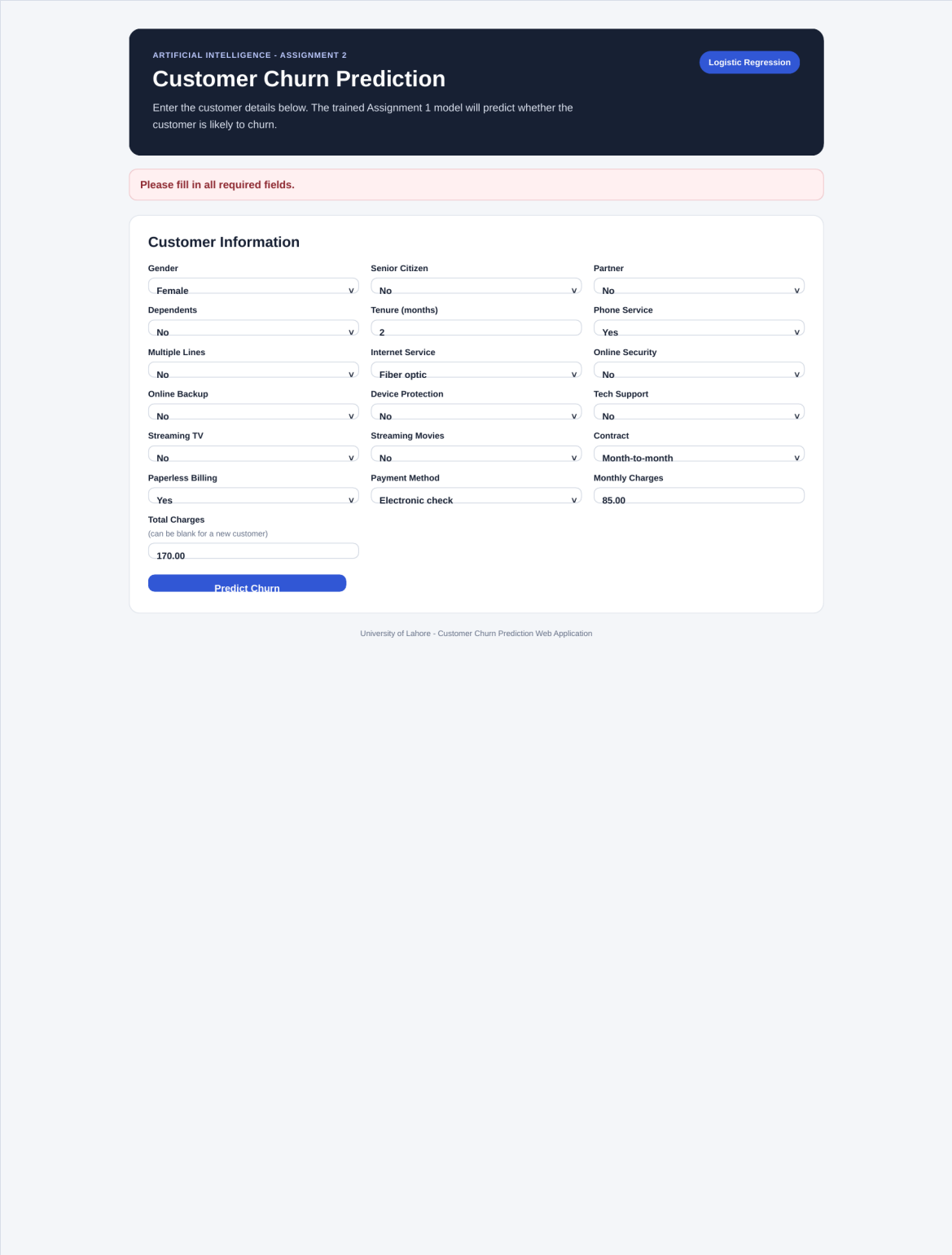
The second sample is a long-tenure customer with a two-year contract and support services. The trained model gives a very low churn probability.



Screenshot 8 - No Churn case. The result is **No Churn** with a churn probability of about **0.7%**.

10. Error Handling

The application checks the submitted values before prediction. If a required field is missing or a numeric value is invalid, the page shows a normal message rather than crashing.



Screenshot 9 - Validation message. A user-friendly error is shown when required information is incomplete.

11. PythonAnywhere Deployment

The project already contains a PythonAnywhere WSGI configuration example and a complete `requirements.txt`. The WSGI path only needs the actual PythonAnywhere username before deployment.

PythonAnywhere WSGI Configuration Example

```
# Replace YOUR_USERNAME with your PythonAnywhere username.
import sys

project_home = '/home/YOUR_USERNAME/customer_churn_flask'
if project_home not in sys.path:
    sys.path.insert(0, project_home)

from app import app as application
```

Screenshot 10 - WSGI configuration example. The project folder is added to the Python path and the Flask application is imported as `application`.

> **Deployment evidence still required:** PythonAnywhere requires the student's own account. After uploading the ZIP, the final university submission must include (1) the actual PythonAnywhere Web/WSGI configuration screenshot, (2) a screenshot of the public deployed website, and (3) the working live URL.

Live URL: _____

12. Conclusion

Assignment 2 converts the Assignment 1 churn model into a usable web application. The model and preprocessing are stored together, the Flask route accepts customer data, the prediction helper keeps the same feature order, and the page displays an understandable result. Local model tests produced both a Churn and a No Churn case, and validation was added for incomplete or incorrect inputs.

Files ready for submission

- Complete Flask project folder / ZIP
- `model.joblib` and `model_metadata.json`
- HTML and CSS frontend
- `requirements.txt`
- Local testing script
- PythonAnywhere deployment instructions and WSGI example
- This PDF report with screenshots

> **Before submitting:** deploy the project on PythonAnywhere, test the public URL, then add the real deployment screenshots and URL to this report. This step should not be replaced with a fake URL or a made-up screenshot.